MDPI

Article

From Prompt 2 Master Plan: Towards Solving Ill-Defined Problems with LLM-MAS

Vasilii Starikov [1]* 0009-0002-8206-7753, Igor Zvyagin [1] 0009-0005-9811-5459, Maksim Natykin [1] 0009-0003-8195-0007, Tatiana Churiakova [1] 0000-0001-7945-8150 and Sergey Mityagin [1] 0000-0001-9877-1687

[1] AI Institute, ITMO University, Saint Petersburg 197101, Russia
* Correspondence: vasilstar97@gmail.com

Highlights

What are the main findings?

- FP2MP, a blackboard-based multi-agent LLM architecture, solves ill-defined urban planning problems via iterative uncertainty reduction, achieving superior metacognition (5.00 ± 0.00 vs. 2.26-3.51 for baselines) and uncertainty handling (4.74 ± 0.45 vs. 2.46-3.83) across 60 problem instances. The FP2MP-Eval protocol demonstrates high inter-rater reliability and sensitivity to model capability differences, enabling multi-dimensional assessment of problem structuring, solution quality, and reflective reasoning without ground-truth answers.

What is the implication of the main findings?

- The proposed framework introduces FP2MP, a blackboard-based LLM-MAS architecture that operationalises evidence-based ill-defined urban planning problem solving by combining tool-augmented agents with a shared three-level memory and critic-driven closed-loop control. Proposed FP2MP-Eval evaluation framework provides a multi-dimensional LLM-as-a-judge evaluation protocol for ill-defined tasks, enabling reproducible assessment of problem structuring, solution quality, and reflective reasoning without ground truth. The open-source implementation (core architecture, evaluation, and baselines) offers a reusable foundation for developing and benchmarking LLM-MAS on ill-defined problems beyond urban planning.

Abstract

Ill-defined urban planning problems, such as master plan generation, involve conflicting objectives, heterogeneous data sources, ambiguous evaluation criteria, and no single correct solution. Existing LLM-based multi-agent systems (LLM-MAS) are evaluated primarily on well-defined benchmarks with ground-truth answers, creating a fundamental mismatch between system capabilities and real-world decision-making needs. We introduce FP2MP (From Prompt 2 Master Plan), a blackboard-based multi-agent architecture that operationalizes ill-defined problem-solving as iterative uncertainty reduction through a closed-loop workflow: decomposition (ReDI), execution via specialized tool-augmented agents (web search, normative retrieval, geospatial computation), curation with evidence promotion, and critic-driven follow-up injection. The system maintains a three-level shared memory (raw data → wiki pages → confirmed facts), enabling adaptive collaboration depth without fixed interaction rounds. We also propose FP2MP-Eval, a multi-dimensional LLM-as-a-judge evaluation protocol assessing problem structuring, solution quality, and reflective reasoning across eight criteria (5-point Likert). Validation on 30 ill-defined urban planning tasks demonstrates high inter-rater reliability (ICC > 0.82) and sensitivity to

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Smart Cities* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

model capability differences. Quantitative comparison against six baselines (Single-Agent, Chain-of-Thought, ReAct, Actor-Critic, Majority Voting, Multi-Agent Debate) on 60 problem instances shows that FP2MP achieves superior performance in metacognition (5.00 ± 0.00 vs. 2.26–3.51 for baselines) and uncertainty handling (4.74 ± 0.45 vs. 2.46–3.83), while remaining competitive in coherence and decomposition. A case study on Gatchina, Russia, demonstrates that FP2MP proposes a polycentric, evidence-based master plan grounded in verifiable spatial computations (e.g., clinic provision improvement from 0.295 to 0.833). The open-source implementation supports reproducibility and provides a foundation for LLM-MAS evaluation on ill-defined problems beyond urban planning.

Keywords: urban planning, large language models, multi-agent systems, ill-defined problems

1. Introduction

Urban planning involves complex decision-making processes that operate under multiple conflicting objectives such as sustainability, mobility, economic development, and social equity. Problems in this domain include transport network design [1,2], infrastructure placement [3,4], and land use allocation [5,6], all of which require reasoning over heterogeneous constraints and stakeholder preferences.

In practice, many urban planning problems are weakly formalized and can be considered inherently ill-defined: they do not admit a single correct solution, lack a fixed procedural pipeline, and do not have a clearly specified objective function [7]. Moreover, such problems require the integration of multimodal data sources, including spatial and statistical data, analytical reports, regulatory and urban planning documents, as well as common knowledge that is difficult to fully formalize or represent in structured form. In addition, the difficulty of collecting, integrating, and processing these heterogeneous data sources further increases the computational and operational complexity of decision-making processes.

At the most comprehensive scale, the construction of an urban master plan can be viewed as a highly complex instance of such ill-defined decision-making, involving heterogeneous data sources, conflicting stakeholder objectives, ambiguous and evolving goals, and uncertainty regarding both appropriate methodological approaches and evaluation criteria [8].

This setting creates opportunities for large language models (LLMs) and multi-agent systems (LLM-MAS) built upon them, which have demonstrated strong capabilities in reasoning, planning, and tool use, making them promising candidates for supporting complex decision-making tasks. These capabilities are enabled by architectural properties such as role-based decomposition, iterative refinement, and inter-agent interaction [9,10].

However, their evaluation is still predominantly conducted on well-defined benchmarks with explicit ground truth and objective scoring functions [11,12]. This creates a fundamental mismatch between real-world ill-defined decision-making problems and existing evaluation paradigms for LLM-based systems. Current evaluation methods provide limited insight into the multi-dimensional nature of solution quality and often fail to capture or diagnose specific strengths and weaknesses of LLM-based system behaviors [7].

This creates a fundamental mismatch between real-world ill-defined decision-making problems and existing evaluation paradigms for LLM-based systems. Current evaluation methods provide limited insight into the multi-dimensional nature of solution quality and often fail to capture or diagnose specific strengths and weaknesses of LLM-based systems behaviors.

Despite growing interest in LLM-based agents, evaluation frameworks in ill-defined settings remain relatively underexplored, particularly in the context of urban planning. We address these limitations by making the following contributions:

- **FP2MP**: an LLM-MAS architecture for solving ill-defined urban planning problems.
- **FP2MP-Eval**: a multi-dimensional LLM-as-a-judge evaluation framework designed to assess reasoning quality, structuring ability, and reflective behavior of LLM-MAS in ill-defined settings.

2. Related Work

2.1. AI in Urban Planning

Cities are dynamic systems influenced by a wide range of interacting factors. In large urban environments, multifunctional activities often coexist, and their interactions causes complex and conflicting urban planning challenges.

Large language models can serve as intelligent assistants for urban planners, supporting the synthesis of conceptual ideas, generation of urban design proposals, and evaluation of planning outcomes [13]. Multi-agent LLM approaches further enable effective handling of complex urban planning tasks while maintaining high relevance of generated outputs, making them practical for real-world applications in urban governance [14].

Existing digital planning solutions often struggle to adapt to the rapidly changing nature of urban environments and are typically designed for single datasets or specific use cases, which limits their generalizability across different planning tasks [15]. In [16], agent-based approaches are explored for urban planning, focusing on optimizing basic infrastructure requirements at the neighborhood level to achieve balanced and coherent urban development. The authors of [17] aim toward more transparent city management. Their approach leverages pretrained end-to-end optimization models and stakeholder-oriented natural language query features to ensure robust performance under uncertainty. PlanGPT [18] successfully addresses challenges in urban planning document management, review, and assessment, significantly improving timeliness and multimodal handling of planning artifacts. Incorporating demographic and life-value data, the work in [19] proposes a predictive framework for anticipating stakeholder responses via simulation-based modeling, thereby enabling more equitable and cost-effective urban planning decisions. The study in [20] investigates transparent AI deployment in urban planning and establishes systematic design principles supporting sustainable urban development through effective human–AI collaboration.

Traditional approaches rely heavily on extensive historical data and assume relatively stable relationships between variables, whereas LLM-based systems demonstrate strong performance even in low-shot or zero-shot forecasting scenarios by leveraging deeper contextual understanding and interdisciplinary knowledge [21]. However, LLMs often struggle with heterogeneous data streams originating from multiple sources, especially when these span different spatial resolutions and temporal scales [22]. To address this challenge, CityGPT [23] introduces a systematic framework for evaluating and enhancing the spatial cognition abilities of general-purpose LLMs in urban environments. Another approach, ShapefileGPT [24], effectively processes complex vector geospatial data, demonstrating strong spatial reasoning and analytical capabilities.

In [25], a comparison is conducted between human-generated plans and ComPlanAI, a hybrid AI system combining rule-based computation with large language models for comprehensive urban planning. The results highlight the potential of AI-assisted co-design in strategic planning contexts. However, more comprehensive evaluation of planning quality requires integration of cartographic and GIS-based data into assessment frameworks. The study in [22] addresses this limitation by transforming natural language instructions

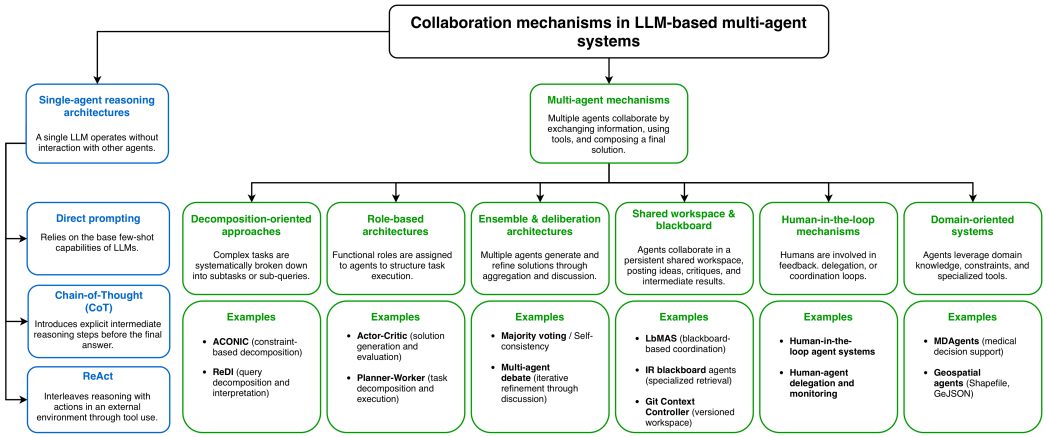


Figure 1. Collaboration mechanisms in LLM-based multi-agent systems PLACEHOLDER

into structured GeoJSON operations using LLM enhancement techniques such as function calling and code generation.

2.2. Collaboration Mechanisms in LLM-MAS

Collaboration mechanisms in LLM-based multi-agent systems (LLM-MAS) define how agents decompose tasks, exchange intermediate results, use tools, refine hypotheses, and compose a final solution. Contemporary surveys typically examine these mechanisms along several dimensions: the number and types of agents, communication structure, coordination protocol, role specialization, and solution aggregation methods [26]. We categorize collaboration mechanisms in LLM-MAS into several major groups as shown in Figure 1.

2.2.1. Single-Agent Reasoning Architectures

The simplest reasoning setting involves a single LLM operating without interaction with other agents. This group includes direct prompting approaches, which rely on the base few-shot capabilities of large language models [27]. A second line of methods extends single-model reasoning through structured intermediate computation. Chain-of-Thought prompting introduces explicit intermediate reasoning steps before producing a final answer [28], while ReAct interleaves reasoning with actions in an external environment through tool use [9]. These approaches cannot be considered multi-agent collaboration mechanisms, but they serve as lower baselines before introducing inter-agent communication.

2.2.2. Role-Based Architectures

The second group introduces explicit functional role separation between agents. In actor–critic approach, one agent produces an initial solution, while another evaluates it, identifies errors, incompleteness, or weaknesses, and provides feedback for refinement. This logic is related to self-refinement approaches, where a model iteratively improves its own response based on feedback [29], as well as to contemporary actor–critic frameworks aimed at improving the reliability of LLM agents in multi-step scenarios [30]. A related pattern is the planner–worker architecture: the planner decomposes the task and manages the execution process, while worker agents solve specialized subtasks. This approach is often used in domain-oriented systems, including geospatial agents for processing Shapefile and GeoJSON data [22,24]. Such architectures are effective when a task can be represented as a sequence of explicit operations, but their communication structure is typically predefined and centralized.

2.2.3. Ensemble and Deliberation Architectures

The third group includes ensemble-style and deliberation-based coordination protocols. Majority voting and self-consistency aggregate several independently generated solutions, reducing the dependence of the result on a single stochastic generation trajectory [31]. Multi-agent debate extends this idea: multiple agents first independently generate answers, then observe the answers of other agents and refine their own solutions over several rounds [32]. However, contemporary research shows that a significant portion of the improvements attributed to debate approaches may be explained not by the debate itself but by the ensemble effect of voting [33].

2.2.4. Shared Workspace and Blackboard Systems

A more flexible group of mechanisms is inspired by collaborative problem-solving systems used in real-world settings involving open-ended and ill-structured tasks. Blackboard architectures assume a shared workspace in which specialized agents can post partial solutions, facts, hypotheses, criticisms, and intermediate conclusions. The classic blackboard model was developed for complex, ill-structured problems requiring the integration of heterogeneous sources of expertise [34]. In LLM-MAS, this idea is adapted by coordinating agents not only through fixed rounds of discussion but also through persistent collective memory. LbMAS follows this direction: agents are selected based on the current state of the blackboard, and the process continues until a consensus solution is reached [35]. Similar blackboard-based systems have also been proposed for information retrieval tasks, where agents autonomously respond to queries in a shared workspace according to their specialization [36]. Another related direction is context management via a version-controlled workspace: the Git Context Controller treats agent memory as structured context with commit, branch, merge, and context retrieval operations, making long agentic workflows more manageable and reproducible [37]. Such approaches are particularly important for open-ended tasks where the solution cannot be reduced to selecting a single correct answer from a fixed set of options.

2.2.5. Decomposition-Oriented Approaches

ACONIC proposes systematic decomposition of complex LLM tasks by formalizing them as constraint satisfaction problems. The framework uses formal complexity measures to determine suitable decomposition structures [38]. ReDI applies decomposition and semantic interpretation to complex search queries: the original user query is broken down into sub-queries, each sub-query is augmented with an interpretation, and then the search results are aggregated [39]. Although such works are not always multi-agent systems in the strict sense, they provide useful principles of decomposition, interpretation, and aggregation that can be embedded into planner-based and blackboard-based LLM-MAS.

2.2.6. Human-in-the-Loop and Domain-Oriented Systems

Domain-oriented architectures take into account that solution quality depends not only on the model's general reasoning ability but also on expert knowledge, domain constraints, and correct interpretation of context. For example, MDAgents dynamically selects an individual or group interaction structure depending on the complexity of a medical task, simulating real clinical decision-making processes [40]. Human-agent systems add another level of control by incorporating a human into delegation, monitoring, feedback, or coordination loops [41]. This is especially important for ill-structured decision-support tasks, where the result must be not only formally correct but also interpretable, contextually justified, and aligned with stakeholder constraints.

2.2.7. Summary

Overall, existing collaboration mechanisms in LLM-MAS have been developed primarily for question answering, reasoning, information retrieval, coding, and domain benchmark tasks, where an objectively correct or externally verifiable answer is assumed. For ill-structured applied tasks, including urban planning decision support, the most promising direction is a combination of tool-augmented reasoning, shared workspace mechanisms, explicit decomposition, and role specialization. Among these, blackboard- and context-management-based mechanisms are particularly relevant, as they support incremental construction of solutions and coordination of heterogeneous agents without requiring a rigid interaction protocol. Human-in-the-loop integration is an important and promising direction for such systems, however, it remains outside the scope of this work and is left for future investigation.

2.3. Evaluation of LLM-MAS

Evaluation of large language models and LLM-MAS spans a wide range of benchmarks designed primarily for well-defined problems. These benchmarks cover diverse domains, including:

- Question answering: Qasper [42], MuSiQue [43], NarrativeQA [44], 2WikiMQA [45].
- General knowledge evaluation: MMLU [11].
- Reasoning tasks: ARC-Challenge [46], GPQA [47], BIG-Bench HARD [48].
- Mathematical problem solving: GSM8K [12], MATH [49].
- Domain-specific applications: MedQA [50], PubMedQA [51], DDXPlus [52], MIMIC-CXR [53], PlanBench [54].

In addition to general-purpose benchmarks, many application domains define custom evaluation datasets. These include datasets for spatial analysis tasks [22,24], as well as expert-curated question answering datasets grounded in domain knowledge [14].

Despite their diversity, these benchmarks are primarily designed for well-defined tasks with fixed input-output structure and existing ground-truth. Evaluation in this setting is typically objective and relies on exact match, multiple-choice accuracy, or other deterministic scoring functions. This paradigm, however, does not generalize well to ill-defined problems where multiple valid solutions may exist and evaluation requires subjective judgment over reasoning quality and solution adequacy.

To address these limitations, recent LLM-as-a-judge approaches such as G-Eval [55] and CheckEval [56] propose decomposing evaluation into structured, predefined criteria, enabling more flexible and interpretable evaluation protocols.

Recent work has also begun exploring evaluation settings for ill-defined problems. PMC [57] introduces a large-scale benchmark of mathematical problems with missing or contradictory conditions, explicitly targeting ill-defined scenarios where tasks may not admit a valid solution. Their results demonstrate that conventional reasoning methods exhibit a trade-off between solving accuracy and the ability to recognize unsolvable problems. More broadly, many modern LLM evaluation settings are inherently ill-defined [7]. Existing scalar evaluation metrics often fail to provide interpretable diagnostic signals, highlighting the need for multi-dimensional evaluation protocols capable of capturing different aspects of solution quality.

2.4. Gap and Positioning of This Work

Existing research has demonstrated significant progress in LLM-based multi-agent systems and their coordination mechanisms, including architectures for reasoning and collaborative problem solving. However, the evaluation of such systems remains largely grounded in well-defined benchmark setting with explicit ground-truth signals and objec-

tive correctness criteria. This creates a fundamental mismatch between the capabilities of modern LLM-MAS and the evaluation protocols used to assess them.

This limitation is particularly pronounced in urban planning, where problems are underspecified, admit multiple valid solutions and require trade-offs between competing objectives. Existing approaches either reduce problems to structured subtasks or rely on heuristic evaluation that do not capture the full complexity of generated solutions.

In this work, we address this gap by proposing:

- **FP2MP**: an LLM-MAS architecture for solving ill-defined urban planning problems inspired by LbMAS architecture [10].
- **FP2MP-Eval**: a multi-dimensional LLM-as-a-judge evaluation framework for assessing reasoning quality, problem structuring, and reflective capabilities of LLM-MAS in ill-defined settings.

3. Methodology

3.1. Problem Formulation

We model the generation process of solutions for ill-defined urban planning problems as a stochastic function:

$$s_{p,q} \sim M_q(p)$$

where:

- M_q denotes a language model-based baseline method with fixed decoding parameters,
- $p \in P$ denotes an ill-defined urban planning problem,

Due to computational constraints, we draw a single sample per (p, M_q) pair. Thus, each observed solution corresponds to one realization from the underlying generation distribution induced by the baseline method.

3.2. Problems Construction

We construct a benchmark of ill-defined urban planning problems by combining a set of urban locations $L = \{l_1, l_2, \ldots, l_n\}$ with a set of ill-defined urban planning questions $Q = \{q_1, q_2, \ldots, q_m\}$. The set of urban locations is presented in Table 1 and the set of ill-defined questions is presented in Table 2.

Problems are generated using a template function $T(l, q)$, where T constructs a natural language prompt by concatenating a location and a question using a fixed prompt template (see Appendix A.1):

$$P = \{T(l, q) \mid l \in L,\ q \in Q\}$$

The resulting benchmark consists of $|P| = |L| \cdot |Q| = n \cdot m$ problem instances. Each problem instance $p \in P$ can be indexed as $p_{i,j} = T(l_i, q_j)$.

3.3. FP2MP LLM-MAS Architecture

The proposed method frames weakly structured problem solving as an *iterative uncertainty reduction process* over a shared state space, rather than a single-shot artifact generation task. Architecturally, the system is implemented as a computation graph (LangGraph) built on top of a three-level blackboard memory and controlled by a closed-loop *decomposition–execution–curation–critique* cycle (see Figure 2).

A key design principle is that the properties distinguishing LLM-based multi-agent systems (multi-agent coordination, reasoning, tool use, iterative refinement, aggregation,

ID	Location	Country	Continent	Population	Justification
l_1	Amsterdam	The Netherlands	Europe	0.9 million	A European capital with access to the sea and a population of about one million
l_2	Singapore	Singapore	Southeast Asia	6.2 million	An island country and city-state in Southeast Asia with a population of over five million
l_3	Kemerovo	Russia	Asia	0.5 million	A landlocked city in Siberia with a population of up to one million people
l_4	Helsinki	Finland	Europe	0.7 million	A European capital with access to the sea and a population of up to one million
l_5	Tashkent	Uzbekistan	Central Asia	3.2 million	Asia's oldest landlocked city with a population of over one million
l_6	Minsk	Belarus	Europe	2.0 million	A landlocked European capital with a population of over one million

Table 1. Urban locations used for problems construction.

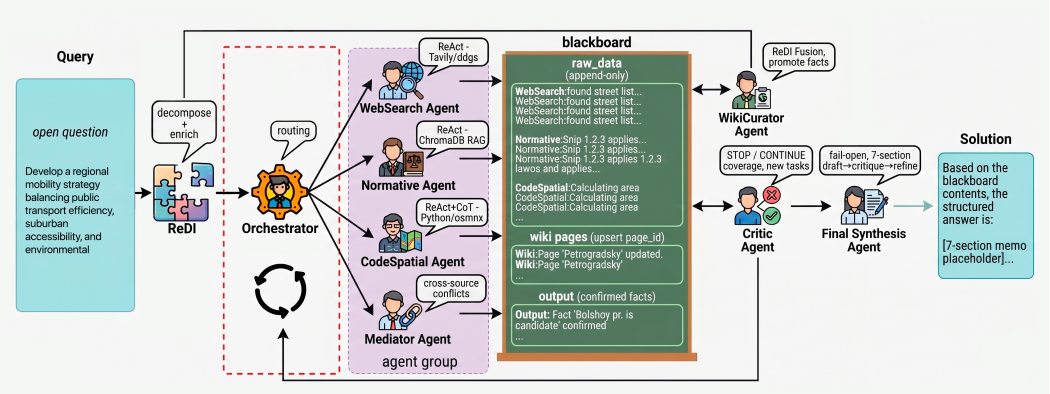


Figure 2. Proposed FP2MP LLM-MAS architecture.

communication, and shared memory) are not treated as separate modules. Instead, they emerge naturally from a unified graph structure.

3.3.1. Query Decomposition (ReDI)

Given an ill-defined problem p, the ReDI module decomposes it into a set of independent sub-queries:

$$D(p) = \{(sq_i, m_i)\}_{i=1}^{k}, \qquad k \in [3,5],$$

where each sub-query sq_i is assigned an intent aspect and a modality hint $m_i \in \{\text{web, normative, code, any}\}$.

During enrichment, each sq_i is augmented with paraphrases, keywords, and domain-specific prompts to improve retrieval coverage. Importantly, modality is used only as a *routing context* rather than a strict constraint: final executor selection remains the responsibility of the orchestrator.

ID	Question	Justification
q_1	Develop a regional mobility strategy balancing public transport efficiency, suburban accessibility, and environmental sustainability.	Infrastructure development objectives (transport, greenery, accessibility)
q_2	Design a waterfront redevelopment strategy resilient to climate risks and attractive for mixed urban uses.	Challenges of spatial development taking into account climate change
q_3	Revitalize shrinking neighborhoods while preventing social displacement and economic decline.	Challenges of spatial development according to the goals of socio-economic development
q_4	Create a governance framework for coordinating fragmented metropolitan municipalities.	Challenges of city management, project implementation and operation
q_5	Reduce spatial inequality in access to green spaces, education, and healthcare services.	Infrastructure development objectives according to the goals of socio-economic development
q_6	Increase urban resilience to demographic aging while maintaining economic productivity and social vitality.	Objectives of the strategy for the economic development of the city while preserving the population
q_7	Transform abandoned industrial districts into productive urban spaces.	Challenges of spatial development with economic development
q_8	Design a downtown densification strategy while preserving livability.	Objectives of the spatial compression strategy based on infrastructural diversity
q_9	Develop a city branding strategy balancing tourism and local identity.	Objectives of attracting investments and finding a balance between the interests of local residents and tourists
q_{10}	Propose a strategy for the preservation or renewal of cultural heritage sites.	Challenges of architectural and planning solutions

Table 2. Ill-defined urban planning questions used for problems construction.

3.3.2. Shared Memory (Blackboard)

System state is represented as a three-level shared blackboard:

$$B = (R, W, O),$$

where:

- R — raw outputs log (`raw_data`, append-only),
- W — structured wiki-like knowledge pages (upsert by `page_id`),
- O — validated facts (upsert by `fact_id`, conflict resolved by confidence).

Agents do not communicate directly. Instead, they interact via read and write operations on the blackboard. This design simultaneously implements shared memory and indirect communication, while allowing the solution path to evolve dynamically as new evidence is added.

3.3.3. Orchestration and Specialized Agents

The orchestrator performs capability-based routing. Using structured capability cards (`handles/cannot_do`), it assigns each sub-query to the most suitable agent via LLM-based reasoning.

Execution is parallelized (up to three dispatches per round), and failed tasks are re-routed if confidence drops below 0.4 or an agent exceeds iteration limits.

The system includes the following specialized ReAct agents:

- **WebSearchAgent**: web retrieval (Tavily/DDGS) and page extraction for factual queries.
- **NormativeAgent**: retrieval from a local ChromaDB-based normative index with web fallback.
- **CodeSpatialAgent**: Python-based execution for spatial and numerical reasoning (osmnx, geopandas, geopy).

The CodeSpatialAgent follows a strict pipeline:

$$\text{HYPOTHESIS} \rightarrow \text{PLAN} \rightarrow \text{CODE} \rightarrow \text{EXECUTE} \rightarrow \text{INTERPRET},$$

ensuring deterministic computation instead of textual approximation, which reduces spatial hallucinations.

3.3.4. Curation, Aggregation, and Critique

The **WikiCurator** acts as a fan-in aggregation node. It transforms raw outputs into structured wiki pages and performs *ReDI Fusion*, which includes:

- deduplication via Jaccard similarity,
- keyword-overlap-based reranking,
- promotion of reliable statements into validated facts.

A fact f is promoted into O if:

$$\text{conf}(f) > \tau \ (\tau = 0.65) \quad \text{or} \quad |\text{sources}(f)| \geq 2.$$

The **MediatorAgent** performs cross-source synthesis, identifying agreements and contradictions. The **CriticAgent** evaluates coverage of sub-queries and decides between $STOP$ and $CONTINUE$, optionally injecting missing tasks. Termination occurs when all sub-queries are covered or a maximum iteration limit $T_{\max}$ is reached.

The system transition is defined as:

$$B_{t+1} = \text{Curator}\big(\text{Agents}(\text{Orch}(B_t))\big),$$

with stopping condition:

$$\text{Critic}(B_t) = \text{stop}.$$

3.3.5. Final Synthesis (Fail-Open Mode)

After termination, the system produces a final answer in *fail-open* mode: it always outputs a concrete decision (e.g., choice, location, numeric value, or recommendation) instead of deferring with uncertainty.

The synthesis is based on weighted aggregation of validated facts and source agreement. Reasoning quality is ensured via a three-pass DRAFT $\rightarrow$ CRITIQUE $\rightarrow$ REFINE pipeline. The final output is structured as a seven-section expert memo:

1. directed answer,
2. problem framing,
3. key dependencies,
4. alternatives,
5. justification,
6. residual uncertainty,
7. self-reflection.

3.4. Validation Setup

For validation experiments, we use the deepseek-v4-flash large language model across all agents and baselines, with a fixed temperature of 0.5 unless otherwise specified. All baselines and FP2MP implementations are developed using the LangGraph and LangChain Python libraries.

FP2MP operates with a global budget of up to three iterations ($max_iterations = 3$). For executor agents, we set the following parameters:

- **WebSearchAgent**: $temperature = 0.1$, $max_iterations = 6$;
- **NormativeAgent**: $temperature = 0.0$, $max_iterations = 8$;
- **CodeSpatialAgent**: $temperature = 0.0$, $max_iterations = 8$.

We evaluate all systems using the proposed FP2MP-Eval protocol (Section 4) as the primary evaluation metric for ill-defined tasks.

3.4.1. Tools Implementation

The tools are implemented as LangChain wrapper functions and attached to the corresponding ReAct executors. The orchestrator, curator, mediator, and critic do not invoke external tools, but instead operate directly on the blackboard state. Each executor is assigned a small, focused set of tools rather than a shared pool, which is precisely what enables capability-based routing. A summary is provided in Table 3.

The implementation of the tools is guided by two design choices. First, every external channel has a fallback mechanism (Tavily → DuckDuckGo, local RAG → web search, E2B → `subprocess`), and when a source becomes unavailable, the tool returns an explicit marker for confidence reduction rather than failing outright. Second, executable code undergoes safety checks, API existence verification, and static analysis with `ruff` and `pyflakes` prior to execution, while web pages are distilled into structured evidence notes rather than treated as raw text. In this way, delegating deterministic operations to executable code and source extraction provides evidence traceability and robustness against hallucinations.

3.4.2. Baselines

We compare FP2MP against six LLM-based baselines of increasing complexity, covering the main architectural families in contemporary LLM-MAS literature: single-agent inference (Single-Agent), explicit reasoning (Chain-of-Thought), reasoning with tool use (ReAct), refinement via a critic (Actor-Critic), ensemble voting (Majority Voting), and deliberative multi-agent collaboration (Multi-Agent Debate). Table 4 summarizes their architectural properties. Single-Agent, Chain-of-Thought, and ReAct are single-agent configurations differing only in reasoning and tool-use components. Actor-Critic adds a two-node generator-critic refinement loop ($max_iterations = 1$). Majority Voting ensembles three independent runs ($num_agents = 3$) with no inter-agent communication. Multi-Agent Debate uses three agents with three communication rounds before final aggregation ($num_agents = 3$, $max_iterations = 3$).

In the ReAct setting, the baseline is additionally equipped with the following tools from Table 3:

- `run_validated_python`,
- `list_available_libraries_tool`,
- `check_available_data_tool`,
- `fetch_url_tool`,
- `web_search_tool`.

Tool	Purpose	Implementation
WebSearchAgent		
`web_search_tool`	Web search over a query	Tavily (advanced) with fallback to DuckDuckGo (`ddgs`); automatic query paraphrasing when results are blocked or empty
`research_url_tool`	Deep reading of 1–3 top links	Main text extraction (`trafilatura/readability`) + LLM distillation into evidence notes (claims, numbers, quotes, `source_url`, constraints); protection against repeated URL reads
NormativeAgent		
`normative_vector_search_tool`	Semantic search over a local corpus of regulations	RAG using ChromaDB; multilingual embeddings `MiniLM-L12-v2`; chunks 800/100, top-k=4; source/page included in metadata
`normative_web_search_tool`	Web search for regulatory documents	Wrapper over `web_search_tool` with domain hints (standards/laws/regulations)
`fetch_url_tool`	Citation verification on a page	Page download and text extraction (first 3000 characters)
CodeSpatialAgent		
`run_validated_python`	Validated Python execution	AST safety pre-check + API existence check + `ruff`/pyflakes; backend E2B sandbox or `subprocess`; allowed: `osmnx`, `geopandas`, `shapely`, `geopy`, `networkx`, `pandas`, `numpy`, `matplotlib`; forbidden: `subprocess/eval/exec/os.system`
`find_capability_tool`	Code recipe retrieval	Extraction of verified capability cards (CARD): libraries, I/O contract, common pitfalls
`inspect_api_tool`	API hallucination control	Signature and docstring inspection of an installed object by dotted name
`check_available_data_tool`	Local data inventory	Listing of files in `DATA_DIR`
`list_available_libraries_tool`	Environment inspection	List of installed libraries by category

Table 3. Tools of FP2MP executor agents.

3.4.3. Source Code

The implementation of FP2MP and all experimental pipelines is publicly available and organized across a set of repositories to ensure reproducibility and modularity. The main entry point is the **FP2MP repository**[1], which contains the full experimental pipeline, including problems construction, baseline execution, and results aggregation. In addition, the project is structured into the following components:

- **fp2mp-core**: implementation of the proposed FP2MP multi-agent architecture.
- **fp2mp-eval**: multi-dimensional LLM-as-a-judge evaluation framework used for scoring model outputs.
- **fp2mp-baselines**: implementations of all baseline LLM and LLM-MAS methods used in experiments.

All repositories follow a unified configuration format and are designed to be interoperable through a shared OpenAI-compatible API interface. Experimental results reported

[1] https://github.com/vasilstar97/fp2mp

Baseline	Multi-agent	Reasoning	Tool calling	Iterative refinement	Aggregation	Communication	Shared memory
Single-agent	×	×	×	×	×	×	×
Chain-of-Thought	×	✓	×	×	×	×	×
ReAct	×	✓	✓	×	×	×	×
Actor-Critic	×	✓	×	✓	×	×	×
Majority Voting	✓	×	×	×	✓	×	×
Multi-Agent Debate	✓	✓	×	✓	✓	✓	×
FP2MP	✓	✓	✓	✓	✓	✓	✓

Table 4. Architectural comparison of baseline methods across key properties of LLM-agent systems.

in this paper can be reproduced by following the setup instructions provided in the main repository.

3.5. Case Study Design

In addition to aggregated benchmark evaluation, we conduct a single, end-to-end, instrumented case study aimed at answering the question of *how* the system actually reasons. Since the master planning task is weakly formalized and lacks a ground-truth solution, the value of the case study lies in evaluating the final output, the traceable reasoning chain, and the supporting evidence.

3.5.1. Motivation

While the proposed FP2MP-Eval evaluation framework captures the distribution of performance quality, it obscures the underlying mechanisms. In contrast, the case study renders the blackboard state, capability routing, and critic loop observable within a single open-ended task. This enables a qualitative explanation of gains in reflective dimensions (problem framing, justification, metacognition, and uncertainty handling) through a concrete, inspectable example rather than relying solely on aggregate numerical scores.

3.5.2. Testbed and Data

The testbed is **Gatchina**, a city in the Leningrad Oblast (28.76 km^2), located 42 km from Saint Petersburg. The city combines a strong historical and cultural structure with a medium scale and a partially mono-functional residential layout, making it a realistic weakly formalized planning context where heritage, spatial, regulatory, and socio-economic evidence must be jointly considered.

The local dataset is based on BlocksNet[2] and consists of 351 blocks (approx. 83,320 inhabitants), a precomputed 351×351 accessibility matrix, and 46 types of urban services. The land use composition includes 211 residential blocks, 50 commercial, 34 recreational, 21 transport-related, 19 industrial, 9 special-purpose, and 7 other blocks.

Open data is obtained from OpenStreetMap via `osmnx` and web search, ensuring that domain-specific knowledge is incorporated into the responses rather than relying solely on general model priors.

3.5.3. Task and Configuration

The **query** is defined as "Create a master plan for the city of Gatchina" as an open-ended formulation with no ground-truth solution and multiple competing objectives (heritage preservation, service accessibility, transport connectivity, and economic development). The base model is gpt-5.5, configured with up to seven iterations ($max_iterations = 7$) and the full FP2MP agent pool (Table 5). The run is executed under the same fail-open setup and expert memo format as the baselines.

[2] https://github.com/aimclub/blocksnet

Agent	Capability	Subquery Modality
BlocksNet	Spatial-quantitative diagnostics over the local city model	model-support
CodeSpatial	Computation and geospatial analysis via executable code	code
WebSearch	Factual and empirical web search	web
Normative	Norms, standards, legal and heritage status	normative

Table 5. FP2MP executor agent pool in the case study run. The orchestrator, mediator, and critic are control nodes and do not invoke tools.

3.5.4. BlocksNet Agent

The main executor in this case study is the domain-specific **BlocksNetAgent**, a ReAct-based urban analytics agent operating over a local digital city model. Unlike the three general-purpose instrumental agents, it is specialized in spatial-quantitative diagnostics and is equipped with approximately thirty tools grouped into families (Table 6). Before computation, the agent is required to produce an `ANALYSIS PLAN` referencing at least one verified capability card (CARD). It concludes its reasoning using a fixed schema: `RESULT` / `REFLECTION` / `HYPOTHESES` / `NUMERIC SELF-CHECK` / `CONFIDENCE` / `LIMITATIONS`. Delegating deterministic operations to executable models rather than textual reasoning ensures traceability of numerical evidence and robustness against spatial hallucinations.

Family	Tools (examples)	Purpose
Capabilities and cache	`find_blocksnet_capability`, `list_service_types`, `list_key_services`, `list_cached_data`, `get_analysis_results`	Retrieval of verified CARD recipes; service inventories; reuse of previously computed results
Model loading	`load_blocks`, `load_accessibility_matrix`, `build_adjacency_graph`	Loading blocks, the 351×351 accessibility matrix, and adjacency graph construction
Accessibility and centrality	`compute_mean_accessibility` (and variants `median/max/area/land_use`), `compute_connectivity`, `compute_population_centrality`, `compute_services_centrality`	Spatial accessibility and network roles of blocks; separation of service and population centers
Services and provision	`compute_service_provision`, `compute_services_count`, `compute_services_density`, `compute_services_collocation`, `compute_shared_provision`, `compute_scenario_provision`	Service deficits under walking-time thresholds and before/after scenario validation
Morphology and land use	`compute_density_indicators`, `compute_development_indicators`, `compute_shannon_diversity`	Built density (FSI/GSI/MXI), development potential, and functional diversity
Block inspection	`get_block_info`	Land-use composition and population at block level
Scenario and optimization	`suggest_target_blocks`, `propose_zone_development`, `optimize_zone_services`	Selection of candidate blocks and generation of development hypotheses (Bayesian optimization via TPE/Optuna)
Visualization	`render_metric_map`	Block-level metric visualizations

Table 6. Domain-specific toolset of the BlocksNet agent (over the local city model), grouped by functional families.

3.5.5. Observation Protocol

The system is fully instrumented: logs capture (i) ReDI decomposition into subqueries with modality annotations and the orchestrator's routing decisions; (ii) every tool invocation with agent identity, tool name, arguments, and observations; (iii) structured agent outputs with confidence scores and blackboard progression status (*promoted*/*not_promoted*); (iv) critic-injected follow-up tasks; and (v) generated artifacts (maps, figures, CSV tables).

4. Evaluation Framework

We introduce FP2MP-Eval, an evaluation protocol for ill-defined problems where no ground-truth solution is available. Instead of comparing outputs to a reference answer, we assess (i) the quality of reasoning and problem handling and (ii) the internal consistency of the generated solutions. To this end, we adopt an LLM-as-a-judge (LLMaJ) approach inspired by prior work on model-based evaluation, in particular G-Eval, which demonstrated the effectiveness of GPT-4-based evaluators [55].

Building on the recent work on evaluating ill-defined tasks in LLMs [7], we move away from collapsing model outputs into a single scalar and instead adopt a multi-dimensional evaluation-paradigm. This reflects the inherent structure of ill-defined problems, where solution quality cannot be fully captured by a single correctness criterion.

From a cognitive perspective, Kitchener's reflective judgement model [58] distinguishes between well-defined and ill-defined problems based on the presence of epistemic monitoring processes, when solving an ill-defined problems requires reasoning about the legitimacy and justification of a solution, rather than merely producing an answer.

This notion is further supported by Schraw and Dennison [59], who describe epistemic monitoring as a metacognitive process in which individuals evaluate and regulate the validity of their own reasoning.

4.1. Evaluation Criteria

The evaluation criteria are designed to capture key aspects of reasoning in ill-defined tasks, including problem structuring, solution generation and reflective reasoning (see Table 7). Each criterion is rated on a 5-point Likert scale:

- 5 - excellent, no major weaknesses
- 4 - good, minor issues
- 3 - acceptable, noticeable gaps
- 2 - poor, major issues
- 1 - very poor or missing

Table 7. Evaluation criteria for ill-defined task assessment

Category	Indicator	Description
Problem Structuring	Framing	Identification of key elements, goals, and assumptions
	Decomposition	Degree to which the problem is structured into coherent subproblems
	Uncertainty Handling	Ability to address ambiguity, missing information, and assumptions
Solution Quality	Coherence	Logical consistency and clarity of reasoning steps
	Justification	Strength of argumentation and consideration of trade-offs
	Knowledge Integration	Use of relevant domain knowledge and contextual information
Reflective Reasoning	Diversity	Exploration of alternative solutions and perspectives
	Metacognition	Reflection on reasoning process and awareness of limitations

4.2. Evaluation Procedure

Following prior LLM-as-a-judge frameworks such as G-Eval, which demonstrated strong evaluation performance using OpenAI's GPT-4, we employ GPT-4.1, a newer model from the same family, for all evaluation runs.

The evaluation framework is implemented using the LangChain Python library. We use a structured output approach in which the evaluation model receives the predefined evaluation criteria (see Appendix B.1) along with a standardized evaluation prompt template (see Appendix B.2). To account for stochasticity in LLM generation while preserving response diversity, the temperature parameter is set to 0.5 for all evaluation runs.

Each problem-solution pair (p, s) is evaluated across m independent runs to reduce possible variance from stochastic LLM generation. Each run produces scores for n evaluation criteria using a 5-point Likert scale. In addition to the score, the evaluator also returns a structured justification consisting of evidence and commentary for each criterion.

Let $C = \{c_1, c_2, \ldots, c_n\}$ denote the set of evaluation criteria. For a given problem p, solution s, and run r, we define the evaluation vector as:

$$\mathbf{x}_r^{(p,s)} = \left[x_{1,r}^{(p,s)}, x_{2,r}^{(p,s)}, \ldots, x_{n,r}^{(p,s)} \right]$$

We aggregate evaluations across runs by computing the mean score for each criterion:

$$\bar{x}_i^{(p,s)} = \frac{1}{m} \sum_{r=1}^{m} x_{i,r}^{(p,s)}$$

The aggregated evaluation vector is defined as:

$$\bar{\mathbf{x}}^{(p,s)} = \left[\bar{x}_1^{(p,s)}, \bar{x}_2^{(p,s)}, \ldots, \bar{x}_n^{(p,s)} \right]$$

4.3. Validation Setup

To assess the stability and adequacy of the proposed evaluation framework, we conducted a validation experiment using LLM-generated solutions for a set of ill-defined urban planning tasks.

We selected three language models representing different capability levels:

- deepseek-v4-pro (DeepSeek) - a strong frontier-level model,
- gpt-4o-mini (OpenAI) - a medium capability model,
- phi-4-mini-instruct (Microsoft) - a lower-capability lightweight model.

The benchmark problem set was constructed by combining a set of 6 urban locations (see Table 1) with the first 5 ill-defined questions (see Table 2). Each problem instance was generated using template function T:

$$p_k = p_{i,j} = T(location_i, question_j)$$

where $location_i$ denotes a location instance, $question_j$ denotes an ill-defined urban planning question, and T constructs a natural language prompt using a fixed template (see Appendix A.1) This resulted in total of $|P| = 6 \times 5 = 30$ distinct ill-defined problems.

For each problem p_k, every model M_q generated an independent solution $s_{k,q}$ resulting in $30 \times 3 = 90$ problem-solution pairs $(p_k, s_{k,q})$.

Each problem-solution pair $(p_k, s_{k,q})$ was evaluated across 5 evaluation runs resulting in $90 \times 5 = 450$ evaluation vectors $x_r^{(p_k, s_{k,q})}$.

The validation pipeline and all experimental artifacts are publicly available in the **FP2MP-Eval** repository[3], allowing full reproduction of the validation setup and results.

4.4. Reliability Analysis

To assess the stability of the proposed evaluation framework, we measure inter-rater reliability using the intraclass correlation coefficient (ICC). ICC is computed separately at the level of problem-solution pairs (p, s) treating multiple evaluation runs $x_r^{(p,s)}$ as different measurements. For each problem-solution pair (p, s) we estimate a set of intraclass correlation coefficients:

$$ICC = \{ICC(1,1), ICC(A,1), ICC(C,1), ICC(1,k), ICC(A,k), ICC(C,k)\}$$

Thus, every problem-solution pair yields a different profile across multiple $ICC = \{ICC_{type}\}$ definitions:

$$ICC^{(p,s)} = [ICC_{type}^{(p,s)}]$$

To obtain dataset-level reliability estimates, we aggregate each ICC type independently across all pairs:

$$\overline{ICC}_{type} = \frac{1}{|P|} \sum_{(p,s)\in P} ICC_{type}^{(p,s)}$$

We additionally report standard deviations across pairs to capture heterogeneity in reliability across different tasks. The resulting aggregated reliability estimates across all problem-solution pairs are reported in Table 8.

	ICC(1,1)	ICC(A,1)	ICC(C,1)	ICC(1,k)	ICC(A,k)	ICC(C,k)
Mean	0.82	0.83	0.87	0.96	0.96	0.97
Std	0.09	0.09	0.06	0.03	0.02	0.02

Table 8. Inter-rater reliability (ICC) of the proposed evaluation framework aggregated across all problem-solution pairs (p, s). Values are reported as mean and standard deviation across pairs.

The results indicate high inter-class reliability across all ICC formulations. Single measure ICC values (ICC(1,1), ICC(A,1), ICC(C,1)) show significant agreement between individual evaluation runs, with values ranging from 0.82 to 0.87. Meanwhile average-measure ICC types (ICC(1,k), ICC(A,k), ICC(C,k)) are higher (0.96-0.97), indicating that aggregating multiple evaluation runs effectively reduces stochastic variance in LLM-based judgments and leads to highly stable aggregated scores. These results suggest that the proposed evaluation framework produces consistent assessments across repeated runs.

This confirms that variability in the evaluation is primarily driven by per-run stochasticity rather than systematic disagreement among evaluations. However, these estimates remain conditioned on the chosen LLM model and may vary under different judge models.

4.5. Validation via Model Comparison

To assess the validity of the proposed evaluation framework, we examine whether it can discriminate between models of different capabilities. We evaluate three models: deepseek-v4-pro, gpt-4o-mini, and phi-4-mini-instruct.

[3] https://github.com/vasilstar97/fp2mp-eval

For each problem-solution pair $(p_k, s_{k,q})$ we obtain an aggregated evaluation vector $\bar{\mathbf{x}}^{(p_k,s_{k,q})}$ as defined in Section 4.2. To obtain model-level scores we aggregate these vectors across all problem-solution pairs associated with the given model M_q:

$$\bar{\mathbf{x}}^{(q)} = \frac{1}{|P_q|} \sum_{(p_k,s_{k,q}) \in P_q} \bar{\mathbf{x}}^{(p_k,s_{k,q})}$$

where P_q denotes the set of problem-solution pairs generated by model M_q. The resulting model-level scores are reported in Table 9

Criterion	deepseek-v4-pro	gpt-4o-mini	phi-4-mini
Coherence	4.23 ± 0.42	3.79 ± 0.41	3.39 ± 0.49
Decomposition	4.25 ± 0.43	3.57 ± 0.50	3.21 ± 0.41
Diversity	3.12 ± 0.63	2.75 ± 0.71	2.67 ± 0.72
Framing	4.23 ± 0.42	3.13 ± 0.36	3.04 ± 0.30
Justification	3.35 ± 0.48	2.46 ± 0.50	2.17 ± 0.38
Knowledge integration	4.32 ± 0.47	3.05 ± 0.28	2.94 ± 0.29
Metacognition	2.35 ± 0.55	1.24 ± 0.44	1.12 ± 0.35
Uncertainty handling	2.67 ± 0.66	1.57 ± 0.66	1.34 ± 0.49

Table 9. Evaluation scores across models of different capability levels. Higher-capacity models consistently receive higher scores across all criteria, indicating sensitivity of the evaluation framework to model quality differences.

We observe a consistent ordering of models across all criteria: deepseek-v4-pro achieves the highest score, followed by gpt-4o-mini and phi-4-mini-instruct. This ordering is preserved across all dimensions, indicating that the framework is sensitive to underlying model capability rather than producing unstructured or noise scores. The largest performance gaps are observed in higher-order reasoning-related criteria, such as knowledge-integration and metacognition, suggesting that the framework is sensitive to differences in reasoning depth and reflective capability. However, diversity exhibits smaller differences between models, indicating that this dimension is less strongly correlated with overall model capability. These results provide evidence for the validity of the proposed evaluation framework.

4.6. Summary

We present a multi-dimensional LLM-as-a-judge evaluation framework for ill-defined tasks, designed to move beyond scalar scoring and ground-truth-based evaluation. The framework reflects key aspects of ill-defined problem solving, including problem-structuring, solution quality and reflective reasoning.

Empirical results demonstrate that the proposed evaluation procedure is both reliable and informative. High inter-rater reliability (ICC) indicates that aggregated LLM-based judgments are stable across repeated evaluation runs, while model comparison results show that the framework consistently discriminates between models of different capability levels.

These results suggest that the framework provides a reliable and informative measurement layer for ill-defined problem settings. In the following sections we build on this evaluation protocol to investigate and compare LLM-MAS approaches for solving ill-defined problems.

5. Results

5.1. Quantitative Results

We report quantitative results across all baselines in terms of (i) multi-dimensional evaluation scores over eight proposed reasoning criteria and (ii) computational cost mea-

sured by token consumption. This analysis allows us to characterize both performance and efficiency trade-offs of different LLM-based approaches to ill-defined problem solving.

5.1.1. Multi-Dimensional Evaluation

We first report multi-dimensional evaluation results across all baselines. Table 10 shows mean scores across eight evaluation criteria along with standard deviation values.

Criterion	Single Agent	Chain-of-Thought	Actor-Critic	ReAct	Majority Voting	Debate	FP2MP
Coherence	4.17 ± 0.38	4.13 ± 0.34	4.44 ± 0.50	4.53 ± 0.50	3.94 ± 0.61	4.17 ± 0.47	4.39 ± 0.52
Decomposition	4.14 ± 0.35	4.09 ± 0.29	4.38 ± 0.49	4.44 ± 0.50	3.89 ± 0.62	4.13 ± 0.43	4.15 ± 0.47
Diversity	3.32 ± 0.62	2.81 ± 0.64	3.41 ± 0.56	3.36 ± 0.55	3.04 ± 0.80	3.10 ± 0.62	3.55 ± 0.54
Framing	4.19 ± 0.39	4.05 ± 0.28	4.44 ± 0.50	4.41 ± 0.49	3.92 ± 0.64	4.15 ± 0.45	4.58 ± 0.49
Justification	3.45 ± 0.50	3.16 ± 0.36	3.91 ± 0.35	3.77 ± 0.48	3.09 ± 0.64	3.37 ± 0.55	4.36 ± 0.56
Knowledge Integration	4.28 ± 0.45	4.13 ± 0.40	4.53 ± 0.50	4.67 ± 0.47	3.97 ± 0.67	4.22 ± 0.50	3.77 ± 0.55
Metacognition	2.60 ± 0.62	2.51 ± 0.59	3.51 ± 0.53	2.98 ± 0.76	2.26 ± 0.87	2.50 ± 0.70	5.00 ± 0.00
Uncertainty Handling	2.80 ± 0.60	3.04 ± 0.54	3.83 ± 0.47	3.24 ± 0.76	2.46 ± 0.86	2.75 ± 0.75	4.74 ± 0.45

Table 10. Multi-dimensional evaluation results across baselines (mean ± std over 60 problem instances), with the highest values underlined.

FP2MP achieves the strongest performance across higher-order reasoning dimensions, particularly in framing, justification, metacognition, and uncertainty handling. Compared to existing baselines, the proposed architecture demonstrates substantially stronger reflective reasoning capabilities, suggesting that blackboard-based multi-agent coordination facilitates deeper analysis of ill-defined problems.

However, simpler baselines such as Single-Agent and Chain-of-Thought perform relatively well on structural dimensions including coherence and decomposition, but exhibit weaker performance on reflective criteria such as metacognition and uncertainty handling. This indicates that direct prompting and linear reasoning strategies are often sufficient for producing locally coherent responses, yet insufficient for handling ambiguity and competing trade-offs characteristic of ill-defined problems.

Among existing multi-agent baselines, Actor-Critic and ReAct achieve consistently strong results across most dimensions. ReAct obtains the highest score in knowledge integration, likely due to its ability to iteratively retrieve and incorporate external information. However, FP2MP substantially outperforms all baselines in metacognitive and uncertainty-aware reasoning, suggesting that proposed coordination mechanism enables more explicit reflection on assumptions, limitations, and alternative perspectives.

Figure 3 further illustrates the qualitative differences between baselines across evaluation dimensions. The radar chart highlights substantial differences in reasoning profiles across baselines. While most methods achieve relatively high scores in coherence and decomposition, only FP2MP consistently maintains strong performance across reflective reasoning dimensions.

However, FP2MP does not achieve the highest score in knowledge integration, where ReAct slightly outperforms other approaches. One possible explanation is that ReAct explicitly externalizes retrieved information through iterative tool-augmented reasoning traces, making domain knowledge more directly observable in the final response. In contrast, FP2MP may internalize parts of the reasoning process within intermediate blackboard interactions and agent coordination steps, while exposing only information directly relevant to the final solution.

5.1.2. Token Consumption

We further analyze the computational cost of different baselines by measuring token consumption across input, output and total tokens. The results are reported in Table 11. Values are reported as mean token consumption over all problem instances. Standard deviation is omitted due to aggregation-level logging constraints.

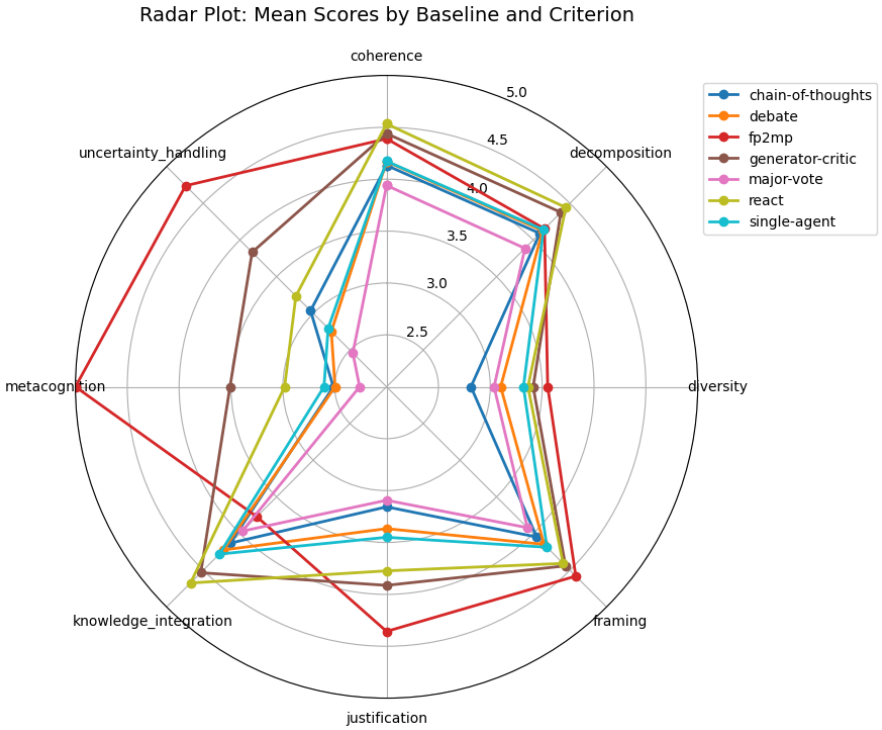


Figure 3. Radar chart of multi-dimensional evaluation scores across baselines. FP2MP demonstrates stronger performance on reflective reasoning dimensions, particularly metacognition and uncertainty handling.

Baseline	Input tokens	Output tokens	Total tokens
Single-agent	20	2130	2151
Chain-of-thought	1444	2904	4349
Actor-Critic	4572	6865	11438
ReAct	292244	11876	304121
Majority Voting	5201	8527	13728
Debate	54285	27717	82003
FP2MP	111323	29150	140474

Table 11. Token consumption across baselines (mean values over 60 problem instances).

The computational analysis reveals substantial differences in cost across baselines. Single-Agent and Chain-of-Thought exhibit the lowest token consumption due to their direct generation structure, while iterative and multi-agent approaches entail significantly higher computational overhead.

Among the evaluated methods, ReAct demonstrates the highest token consumption, primarily due to repeated reasoning-action-observation cycles and extensive interaction with external tools. Debate also require substantial computational resources since multiple agents iteratively refine and exchange intermediate solutions.

FP2MP exhibits higher computational cost compared to simpler baselines, reflecting the complexity of blackboard-based coordination and tool calling mechanism. However, this additional cost is accompanied by improved performance across reflective reasoning dimensions, suggesting a trade-off between computational efficiency and reasoning quality in ill-defined problem solving.

5.2. Qualitative Results

For Gatchina, a polycentric master plan was proposed. Instead of expanding the city through mass housing on the periphery, the approach strengthens existing centers and improves the provision of basic services within residential blocks. This aligns with the

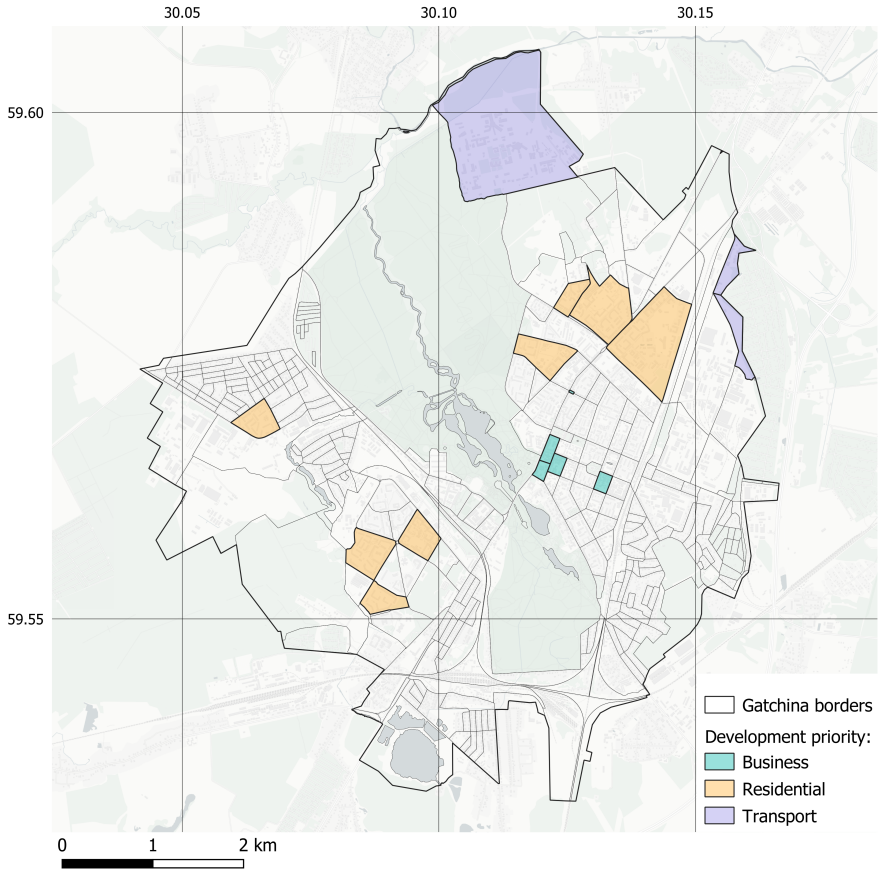


Figure 4. Development priority

concept of sustainable urban development. According to existing protected zones, the model preserves Gatchina's historic park core, while also accounting for view corridors and the continuity of the green infrastructure network. Existing business blocks are proposed to be strengthened, while residential blocks are supplemented with social infrastructure to ensure access to essential services. The master plan strategy is based on the conducted analysis, which revealed a spatial mismatch between residential locations and basic services. The city has a strong transport block with a railway hub; the model proposes reinforcing peripheral interchange nodes as transport blocks 4. The proposals take into account the city's current needs and may serve as a basis for a real-world master plan.

5.2.1. Run Overview

The run was executed on the base gpt-5.5 model with a limit of seven iterations; total runtime was approximately 44 minutes. The system performed exactly 300 tool calls, involving four executor agents, an orchestrator, and a critic. The distribution of calls is strongly asymmetric (see Table 12): 237 out of 300 (79%) were performed by BlocksNet agent, while textual agents (WebSearch, Normative) operated in limited bursts. Importantly, on this ill-defined brief, cooperation did not collapse after the initial coverage of subqueries: computational executors were repeatedly reassigned, and the critic injected an additional round. In other words, the system generated *deep* cooperation that matched the task complexity, exhibiting the behavior this case study was designed to observe.

5.2.2. BlocksNet Agent: Domain Executor

The main executor was **BlocksNetAgent**. Unlike the three general-purpose tool agents, it is specialized in spatial-quantitative diagnostics and includes a suite of more than thirty tools organized into a stable pipeline:

Agent	Calls	Share	Work profile
BlocksNet	237	79.0%	Block-level model diagnostics + development scenarios
WebSearch	38	12.7%	Socio-economic profiling, web search / URL reading
CodeSpatial	14	4.7%	OSM-based geocomputation, executable Python
Normative	11	3.7%	Regulatory corpus search, protected status analysis
Total	**300**	**100%**	

Table 12. Agent load in the case study (number of tool calls in the log).

1. *discovery*: `list_service_types`, `list_key_services`, `list_cached_data`
2. *loading*: `load_blocks`, `load_accessibility_matrix`
3. *computation*: `compute_mean_accessibility`, `compute_service_provision`, `compute_service_provision`, `compute_population_centrality`, `compute_density_indicators`, `compute_shannon_diversity`, `compute_connectivity`
4. *retrieval*: `get_analysis_results`, `get_block_info`
5. *scenario*: `suggest_target_blocks`, `propose_zone_development`, `compute_scenario_provisi`

It is precisely the breadth of this toolset and the block-level granularity (77 calls to `get_block_info`, 54 to `get_analysis_results`) that explain its 79% share of all calls.

The agent's reasoning is structured as a reproducible scheme:

1. `ANALYSIS PLAN`: the agent relies on working cards (CARD `masterplan_baseline_workflow`, `accessibility_network`, `services_diversity_density`, `landuse_optimization`, `service_`) formulates information needs, and justifies each tool selection ("`compute_services_centralit`" and `compute_population_centrality` to separate service centers from population-dense quarters").
2. `REFLECTION`: the agent aggregates and cross-validates results.
3. `HYPOTHESES`: the agent formalizes testable claims in a *claim / test / verdict* format ("claim: adding capacity improves provision; test: `compute_scenario_provision`; verdict: supported"). The agent itself defines the logic: "first measure the baseline state of the city, then propose a testable development hypothesis."
4. `RESULT`: the agent generates final output from the previous steps.

5.2.3. Adaptive Collaboration Depth

The log reflects not a single task allocation but iterative reassignment of executors. Grouping the 300 calls into consecutive single-agent segments yields 17 dispatch segments (see Figure 5). The run begins with a short capability check (CodeSpatial queries available libraries), followed by alternating limited WebSearch and Normative bursts for context collection and large BlocksNet segments (18, 37, 50, 32, 54, 46 calls), into which model-driven diagnostics return six times as the task is progressively refined.

A key episode of collaboration is the injection of an additional round. After identifying in `REFLECTION` a mismatch between the service center (block 136) and the population center (block 331), the system opened follow-up `sq_model_support#fu2`, targeting precisely this uncovered aspect; five of ten structured outputs and the entire second half of BlocksNet segments in Figure 5 are associated with this step. This illustrates adaptive depth: a well-posed query would have been completed in a single pass, whereas this ill-defined brief triggers additional rounds autonomously. Other mechanisms (routing, curation, mediator, critique) are only visible through their effects-reassignment, injection, and promotion of results.

5.2.4. Tool-Grounded Evidence and Context Integration

All reported figures are produced via tool execution rather than model priors. BlocksNet recorded in `REFLECTION/RESULT` an average accessibility of 45.59 minutes (best block

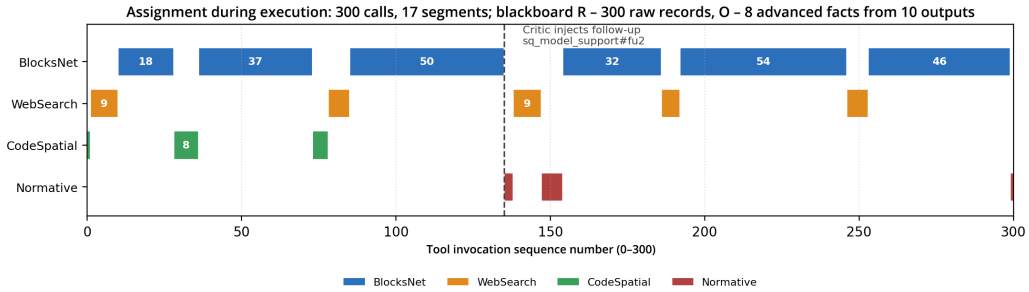


Figure 5. Dispatcher timeline of executors over 300 tool calls (17 segments). BlocksNet returns to execution six times; WebSearch/Normative provide limited context bursts. The dashed line indicates the point after which the critic injected follow-up `sq_model_support#fu2`; this is followed by the second half of BlocksNet segments. The black board accumulates 300 raw records (R), of which 10 structured outputs from executors result in 8 promoted factual statements (O).

196: 32.67 min; worst block 285: 72.01 min) and systemic 15-minute service deficits: clinics — 0.295 (332 out of 351 blocks underserved), supermarkets — 0.288, stadiums — 0.152, local shops — 0.296, while schools — 0.774, pharmacies — 0.821, and stops — 0.962. Service centrality concentrates in block 136 (0.8037), while population centrality peaks in block 331 (1.000); this discrepancy triggered the follow-up. The development scenario (pipeline `suggest_target_blocks` → `propose_zone_development` → `compute_scenario_provision`) for blocks [2, 9, 18, 87, 89, 108, 289, 331] was validated via a before/after comparison: clinics 0.295 → 0.833 (fully served blocks 13 → 86), schools 0.774 → 0.898, pharmacies 0.821 → 0.900, sports facilities 0.460 → 0.545.

Context was provided by other agents: Normative, via web search, established the protected status (the palace and park ensemble as part of a UNESCO object) and SP 42.13330 service radius standards; CodeSpatial extracted the transport framework from OpenStreetMap via the mandatory chain `HYPOTHESIS` → ... → `INTERPRET` (Figure 6); WebSearch collected socio-economic profiles. Only reliable findings are promoted to the shared board: BlocksNet outputs and the completed CodeSpatial transport analysis `sq_003` (confidence 0.78) are marked *promoted*, while incomplete code attempts `sq_002/sq_model_support` (confidence 0.30) remain *not promoted* — 8 out of 10 structured outputs pass the gating mechanism. The observed threshold aligns with the promotion rule, and low confidence is explicitly justified by the agent itself ("numeric self-check missing", "final text did not expose a RESULT field with numbers").

The run also demonstrates robustness to source degradation: WebSearch requests to `ru.wikipedia.org` returned 403 Forbidden, several Normative URLs were broken, and an Overpass query from CodeSpatial was rejected by the server. None of these failures interrupted execution: the system continued using available snippets while preserving a tool-verifiable core. Additionally, an executable-code safeguard triggered a static Pyflakes check (F401) that blocked `run_validated_python` before execution.

5.2.5. Generated Artifacts

The system produces inspectable and reusable outputs, not only text: 71 saved artifacts including 34 CSV tables (block-level service provision, scenario deltas), 19 maps (provision, accessibility, centrality, diversity, density), 7 web-sourced images, and one generated transport network figure. The transport network (Figure 6) and clinic provision map (Figure 7) are the most illustrative, as both directly correspond to key evidence.

5.2.6. Limitations and Threats to Validity

To avoid overinterpreting a single case, we note the following:

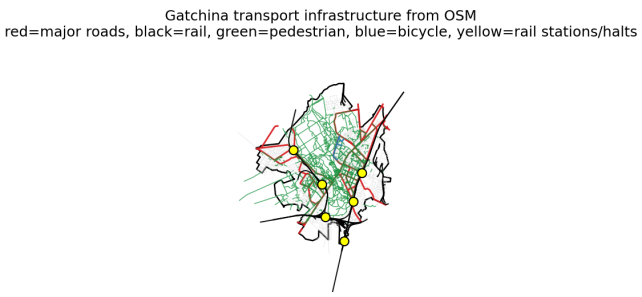


Figure 6. Transport network of Gatchina extracted by CodeSpatial from OpenStreetMap (subquery `sq_003`).

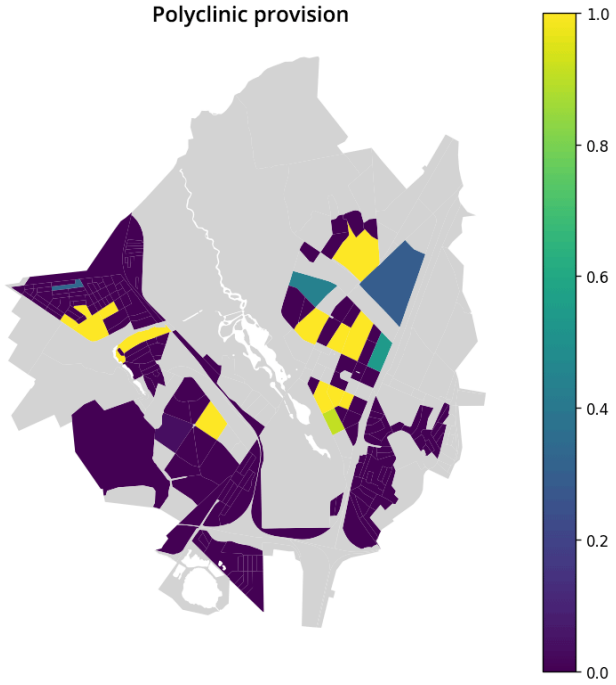


Figure 7. Clinic provision map by block (BlocksNet): strong provision 0.295, with 332 out of 351 blocks underserved. The deficit was identified via `compute_service_provision`.

- This analysis involves a single task, a single base model, and a single run. As such, it should be regarded as an illustration of a coordination mechanism rather than a statistical proof. The demonstration of adaptive depth is based on one complex brief, with no paired simple baseline case for comparison.
- The reasoning chain is constrained by the local BlocksNet model, which includes no up-to-date budgets, land ownership, infrastructure networks, or legal constraints. Furthermore, the optimizer is stochastic, meaning scenario outputs are model-based hypotheses.
- Some external requests failed: 403 Forbidden, broken URLs, and rejected Overpass queries. The system degraded gracefully, but the completeness of web and OSM context is reduced.
- Behavior is observed through execution logs. Internal orchestrator and critic messages are reconstructed from effects (reassignment, follow-up `#fu2`, promotion status), not from a verbatim transcript.

6. Discussion

6.1. Interpretability of Gains

Our quantitative results (see Table 10) show that FP2MP substantially outperforms all baselines in metacognition (5.00 ± 0.00) and uncertainty handling (4.74 ± 0.45), while being competitive in coherence and decomposition. The case study with Gatchina (see Section 5.2) illustrates the mechanism behind this improvement: the three-level blackboard (raw data → wiki pages → confirmed facts) decouples agent communication from fixed debate rounds. Unlike Multi-Agent Debate, where agents exchange messages without persistent memory, FP2MP accumulates verifiable evidence (e.g., `compute_service_provision` showing clinic provision value increase from 0.295 to 0.833). The Critic node then injects follow-up tasks based on gaps (e.g., mismatch between service centrality in block 136 and population centrality in block 331), not on superficial disagreement.

6.2. Mitigating Hallucinations

A notable finding is that FP2MP does not achieve the highest knowledge integration score (3.77 $\pm$ 0.55 vs. ReAct's 4.67 $\pm$ 0.47). We attribute this to a design trade-off: ReAct explicitly externalizes retrieved text, while FP2MP delegates deterministic operations to executable code and tool use (CodeSpatialAgent, BlocksNetAgent). The case study shows 79% of tool calls go to BlocksNet, producing verifiable numeric evidence (e.g., 45.59 min average accessibility). This makes FP2MP more robust to spatial hallucinations but less "verbally rich" in knowledge display.

This design choice also enables extensibility as FP2MP does not require re-implementing domain logic. Any existing urban analytics tool can be integrated as a new agent. The orchestrator will automatically route relevant subqueries, and the blackboard will curate its outputs alongside other agents. BlocksNetAgent serves as a concrete demonstration of this pattern.

6.3. Adaptive Collaboration Depth

Unlike fixed-round debate (3 rounds in our baseline), FP2MP exhibits adaptive depth. In the case study, the system performed 17 dispatch segments over 300 tool calls, with BlocksNet returning six times. The key episode was the critic injecting `sq_model_support#fu2` after detecting an uncovered aspect (service-population centrality mismatch). This demonstrates that the combination of blackboard and critic enables problem-driven computational effort, not a predetermined interaction pattern. For simpler queries, the same loop would terminate earlier. This property is not present in voting or actor-critic baselines.

6.4. Limitations

We acknowledge several limitations:

- **Single case study**: The Gatchina analysis (see Section 5.2) illustrates mechanism but does not provide statistical evidence. The adaptive depth claim rests on one complex brief without a paired simple counterpart.
- **Data preparation overhead**: The BlocksNet agent used in our case study relies on a pre-computed block-network city model. While this model is built from open data sources, replicating it for a new city requires substantial data engineering.
- **External data failures**: In the case study, Wikipedia returned 403 Forbidden, some normative URLs were broken, and an Overpass query was rejected. The system degraded gracefully but with reduced completeness.
- **Evaluation limitations**: Our LLM-as-a-judge approach suffers from two constraints. First, absolute scores are judge-dependent (gpt-4.1), though relative ordering across models is stable. Second, and more fundamentally, the evaluation assesses only the

final solution artifact, not the quality of the underlying reasoning process. Aspects such as how the system navigates ambiguity, revises hypotheses, or recovers from dead ends are not directly captured by the current protocol.

- **Lack of ablation study**: The contribution of individual architectural components has not been isolated, limiting interpretability of the observed performance gains.

7. Conclusions

This paper addressed the mismatch between LLM-MAS capabilities and evaluation paradigms designed for well-defined problems. We proposed FP2MP, a blackboard-based multi-agent architecture for solving ill-defined urban problems. The proposed architecture operationalises problem-solving as iterative uncertainty reduction through decomposition, shared memory, specialised tool-augmented agents, and a critic-driven closed loop.

We also introduced FP2MP-Eval, a multi-dimensional LLM-as-a-judge evaluation protocol for ill-defined settings. The evaluation protocol covers problem structuring, solution quality, and reflective reasoning. Validation confirmed high inter-rater reliability (ICC > 0.82) and sensitivity to model capability differences.

Quantitatively, FP2MP outperformed seven baselines on metacognition (5.00 ± 0.00) and uncertainty handling (4.74 ± 0.45), while remaining competitive on coherence and decomposition. A case study on Gatchina revealed adaptive collaboration depth: 300 tool calls across 17 dispatch segments, autonomous follow-up injection by the critic, and verifiable evidence promotion.

Future work should focus on four directions: systematic evaluation of adaptive depth across diverse problems, ablation studies to isolate component contributions, human-in-the-loop integration for expert oversight, and process-oriented evaluation of reasoning quality.

Author Contributions: Conceptualization, V.S., I.Z. and T.C.; methodology, V.S., I.Z. and M.N.; software, V.S., I.Z. and M.N.; validation, V.S., I.Z. and M.N.; formal analysis, V.S., I.Z. and T.C.; investigation, V.S. and I.Z.; resources, S.M.; data curation, T.C. and I.Z.; writing - original draft preparation, V.S., I.Z., M.N. and T.C.; writing - review and editing, V.S. and S.M.; visualization, V.S., I.Z. and M.N.; supervision, S.M.; project administration, V.S. and S.M.; funding acquisition, S.M. All authors have read and agreed to the published version of the manuscript.

Funding: This work supported by the Ministry of Economic Development of the Russian Federation (IGK 000000C313925P4C0002), agreement No139-15-2025-010.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original data presented in the study are openly available in GitHub at https://github.com/vasilstar97/fp2mp (accessed on 28 may 2026).

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial intelligence
LLM	Large language model
MAS	Multi-agent system
LLM-MAS	LLM-based multi-agent system
LLMaJ	LLM-as-a-judge
LbMAS	Blackboard-based LLM multi-agent system
FP2MP	From prompt 2 master plan

Appendix A Problems construction

Appendix A.1 Problem template

```
PROBLEM_TEMPLATE = "{location}. {question}"
```

Appendix B Evaluation Framework

Appendix B.1 Structured output model

```
from pydantic import BaseModel, Field

class Dimension(BaseModel):
    score : int = Field(ge=1, le=5)
    evidence : str = Field(description='Specific evidence from the
        solution trace supporting the score.')
    commentary : str = Field(description='What is missing or weak.')

class Evaluation(BaseModel):
    framing : Dimension = Field(description='Quality of problem
        framing: identification of key elements, goals, assumptions.')
    decomposition : Dimension = Field(description='How well the
        problem is broken into subproblems and structured.')
    diversity : Dimension = Field(description='Number and quality of
        alternative solutions considered.')
    coherence : Dimension = Field(description='Logical consistency
        and step-by-step reasoning clarity.')
    justification : Dimension = Field(description='Strength of
        argumentation and trade-off analysis.')
    uncertainty_handling : Dimension = Field(description='Handling of
         missing information, assumptions, ambiguity.')
    knowledge_integration : Dimension = Field(description='Use and
        transfer of relevant domain knowledge.')
    metacognition : Dimension = Field(description='Reflection on
        reasoning process and limitations.')
```

Appendix B.2 Evaluation prompt template

```
EVAL_PROMPT = """
You are a strict critic. Avoid giving high scores unless fully
    justified.
You are provided with ill-defined problem and corresponding solution.
Your task is to evaluate the solution quality according to the
    problem.
DO NOT evaluate the problem itself.
---
SCORING GUIDELINES:
- 5 - excellent, no major weaknesses
- 4 - good, minor issues
- 3 - acceptable, noticeable gaps
- 2 - poor, major issues
- 1 - very poor or missing
---
PROBLEM:
```

```
{problem}
---
SOLUTION:
{solution}
"""
```

References

1. Owais, M. Transit network design problem: A half century of methodological research. *Innovative Infrastructure Solutions* **2026**, *11*, 3.
2. Morozov, A.; Kozliak, R.; Kontsevik, G.; Mityagin, S. One Rule to Bring Them All: Investigating Transport Connectivity in Public Transport Route Generation for Equitable Access. *arXiv preprint arXiv:2602.15053* **2026**.
3. Fugaro, S.; SGALAMBRO, A. Multi-objective facility location problems: Systematic literature review and research agenda. *Available at SSRN 5178615* **2025**.
4. Churiakova, T.; Platonov, I.; Bezmaslov, M.; Bikbulatov, V.; Petrosian, O.; Starikov, V.; Mityagin, S.A. Constrained Metropolitan Service Placement: Integrating Bayesian Optimization with Spatial Heuristics. *Smart Cities* **2025**, *9*, 6.
5. Xiao, D.; Kamruzzaman, L.; Vu, H.L. Land use and transport integration across scales: A review of evolving models and strategies for scaling up. *Transportation Research Interdisciplinary Perspectives* **2025**, *32*, 101544.
6. Rahman, M.M.; Szabó, G. Multi-objective urban land use optimization using spatial data: A systematic review. *Sustainable Cities and Society* **2021**, *74*, 103214.
7. Zhou, Y.; Shbita, B. Evaluating Ill-Defined Tasks in Large Language Models. *arXiv preprint arXiv:2603.17067* **2026**.
8. Rittel, H.W.; Webber, M.M. Dilemmas in a general theory of planning. *Policy sciences* **1973**, *4*, 155–169.
9. Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. In Proceedings of the International Conference on Learning Representations, 2023.
10. Han, B.; Zhang, S. Exploring advanced llm multi-agent systems based on blackboard architecture. *arXiv preprint arXiv:2507.01701* **2025**.
11. Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; Steinhardt, J. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* **2020**.
12. Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* **2021**.
13. Zheng, Y.; Xu, F.; Lin, Y.; Santi, P.; Ratti, C.; Wang, Q.R.; Li, Y. Urban planning in the era of large language models. *Nature computational science* **2025**, *5*, 727–736.
14. Kalyuzhnaya, A.; Mityagin, S.; Lutsenko, E.; Getmanov, A.; Aksenkin, Y.; Fatkhiev, K.; Fedorin, K.; Nikitin, N.O.; Chichkova, N.; Vorona, V.; et al. LLM agents for smart city management: Enhancing decision support through multi-agent AI systems. *Smart Cities* **2025**, *8*, 19.
15. Xia, J.; Tong, Y.; Long, Y. Advancements in the application of large language models in urban studies: A systematic review. *Cities* **2025**, *165*, 106142. https://doi.org/https://doi.org/10.1016/j.cities.2025.106142.
16. Singla, P.; Singh, A.; Gupta, A.; Garg, S. Adaptive urban planning: A hybrid framework for balanced city development. *arXiv preprint arXiv:2412.15349* **2024**.
17. Jiao, Z.; Sha, M.; Zhang, H.; Jiang, X.; Qi, W. City-LEO: Toward transparent city management using LLM with End-to-End optimization. *arXiv preprint arXiv:2406.10958* **2024**.
18. Zhu, H.; Zhang, W.; Huang, N.; Li, B.; Niu, L.; Fan, Z.; Lun, T.; Tao, Y.; Su, J.; Gong, Z.; et al. PlanGPT: Enhancing urban planning with tailored language model and efficient retrieval. *arXiv preprint arXiv:2402.19273* **2024**.
19. Gao, J.; Xu, H.; Dao, N.L. Simulating Multi-Stakeholder Decision-Making with Generative Agents in Urban Planning. In Proceedings of the Transdiciplinarity for a Better World: Proceedings of the 32nd International Society of Transdisciplinary Engineering (ISTE) Global Conference, Monterrey, Nuevo Leon, Mexico, 7-11 July 2025. SAGE Publications 1 Oliver's Yard, 55 City Road, London, EC1Y 1SP, 2025, pp. 40–49.
20. Liu, K.; Yigitcanlar, T.; Browne, W.; Fu, Y. Prompts for planning-AI integration: LLM prompt design for supporting sustainable urban development. *Journal of Open Innovation: Technology, Market, and Complexity* **2025**, *11*, 100666. https://doi.org/https://doi.org/10.1016/j.joitmc.2025.100666.
21. Pan, F.; Huang, X.; Bi, Y.; Gao, Y.; Ye, Y.; Wang, H. From tools to partners: How large language models are transforming urban planning. *AI Open* **2025**.
22. Luo, Q.; Lin, Q.; Xu, L.; Wu, S.; Mao, R.; Wang, C.; Feng, H.; Huang, B.; Du, Z. GeoJSON agents: a multi-agent LLM architecture for geospatial analysis—function calling vs. code generation. *Big Earth Data* **2026**, pp. 1–55.

23. Feng, J.; Liu, T.; Du, Y.; Guo, S.; Lin, Y.; Li, Y. Citygpt: Empowering urban spatial cognition of large language models. In Proceedings of the Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2, 2025, pp. 591–602.
24. Lin, Q.; Hu, R.; Li, H.; Wu, S.; Li, Y.; Fang, K.; Feng, H.; Du, Z.; Xu, L. ShapefileGPT: A multi-agent large language model framework for automated shapefile processing. *International Journal of Digital Earth* **2025**, *18*, 2577884.
25. Sung, S.; Jung, C.; Park, Y. Planning with Complanai: Comparative Insights from a Human–Ai Evaluation of Comprehensive Plans. *Available at SSRN 5382891*.
26. Tran, K.T.; Dao, D.; Nguyen, M.D.; Pham, Q.V.; O'Sullivan, B.; Nguyen, H.D. Multi-Agent Collaboration Mechanisms: A Survey of LLMs. *arXiv preprint arXiv:2501.06322* **2025**.
27. Brown, T.B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language Models are Few-Shot Learners. In Proceedings of the Advances in Neural Information Processing Systems, 2020, Vol. 33, pp. 1877–1901.
28. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.V.; Zhou, D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In Proceedings of the Advances in Neural Information Processing Systems, 2022, Vol. 35, pp. 24824–24837.
29. Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; et al. Self-Refine: Iterative Refinement with Self-Feedback. *arXiv preprint arXiv:2303.17651* **2023**, [arXiv:cs.CL/2303.17651].
30. Jiang, S.; Zhang, Z.; Zhang, Y.; Yang, S.; Xia, W.; Soatto, S. Asymmetric Actor-Critic for Multi-turn LLM Agents. *arXiv preprint arXiv:2604.00304* **2026**, [arXiv:cs.AI/2604.00304].
31. Wang, X.; Wei, J.; Schuurmans, D.; Le, Q.V.; Chi, E.H.; Narang, S.; Chowdhery, A.; Zhou, D. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In Proceedings of the International Conference on Learning Representations, 2023.
32. Du, Y.; Li, S.; Torralba, A.; Tenenbaum, J.B.; Mordatch, I. Improving Factuality and Reasoning in Language Models through Multiagent Debate. *arXiv preprint arXiv:2305.14325* **2023**, [arXiv:cs.CL/2305.14325].
33. Choi, H.K.; Zhu, X.; Li, Y. Debate or Vote: Which Yields Better Decisions in Multi-Agent Large Language Models? In Proceedings of the Advances in Neural Information Processing Systems, 2025.
34. Nii, H.P. The Blackboard Model of Problem Solving and the Evolution of Blackboard Architectures. *AI Magazine* **1986**, *7*, 38–53. https://doi.org/10.1609/aimag.v7i2.537.
35. Han, B.; Zhang, S. Exploring Advanced LLM Multi-Agent Systems Based on Blackboard Architecture. *arXiv preprint arXiv:2507.01701* **2025**, [arXiv:cs.AI/2507.01701].
36. Salemi, A.; Parmar, M.; Goyal, P.; Song, Y.; Yoon, J.; Zamani, H.; Pfister, T.; Palangi, H. LLM-Based Multi-Agent Blackboard System for Information Discovery in Data Science. *arXiv preprint arXiv:2510.01285* **2025**.
37. Wu, J.; Hu, M.; Zhu, J.; Pan, J.; Liu, Y.; Xu, M.; Jin, Y. Git Context Controller: Manage the Context of LLM-based Agents like Git. *arXiv preprint arXiv:2508.00031* **2025**.
38. Zhou, T.; Xu, J.; Liu, G.; Liu, J.; Wang, H.; Wu, E. An Approach for Systematic Decomposition of Complex LLM Tasks. *arXiv preprint arXiv:2510.07772* **2025**.
39. Zhong, Y.; Yang, J.; Fan, Y.; Su, L.; de Rijke, M.; Zhang, R.; Cheng, X. Reason to Retrieve: Enhancing Query Understanding through Decomposition and Interpretation. *arXiv preprint arXiv:2509.06544* **2025**.
40. Kim, Y.; Park, C.; Jeong, H.; Chan, Y.S.; Xu, X.; McDuff, D.; Lee, H.; Ghassemi, M.; Breazeal, C.; Park, H.W. MDAgents: An Adaptive Collaboration of LLMs for Medical Decision-Making. In Proceedings of the Advances in Neural Information Processing Systems, 2024.
41. Zou, H.P.; Huang, W.C.; Wu, Y.; Chen, Y.; Miao, C.; Nguyen, H.; Zhou, Y.; Zhang, W.; Fang, L.; He, L.; et al. A Survey on Large Language Model based Human-Agent Systems. *arXiv preprint arXiv:2505.00753* **2025**.
42. Dasigi, P.; Lo, K.; Beltagy, I.; Cohan, A.; Smith, N.A.; Gardner, M. A dataset of information-seeking questions and answers anchored in research papers. In Proceedings of the Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2021, pp. 4599–4610.
43. Trivedi, H.; Balasubramanian, N.; Khot, T.; Sabharwal, A. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* **2022**, *10*, 539–554.
44. Kočiskỳ, T.; Schwarz, J.; Blunsom, P.; Dyer, C.; Hermann, K.M.; Melis, G.; Grefenstette, E. The narrativeqa reading comprehension challenge. *Transactions of the Association for Computational Linguistics* **2018**, *6*, 317–328.
45. Ho, X.; Nguyen, A.K.D.; Sugawara, S.; Aizawa, A. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In Proceedings of the Proceedings of the 28th International Conference on Computational Linguistics, 2020, pp. 6609–6625.
46. Clark, P.; Cowhey, I.; Etzioni, O.; Khot, T.; Sabharwal, A.; Schoenick, C.; Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* **2018**.
47. Rein, D.; Hou, B.L.; Stickland, A.C.; Petty, J.; Pang, R.Y.; Dirani, J.; Michael, J.; Bowman, S.R. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022* **2023**.

48. Suzgun, M.; Scales, N.; Schärli, N.; Gehrmann, S.; Tay, Y.; Chung, H.W.; Chowdhery, A.; Le, Q.; Chi, E.; Zhou, D.; et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In Proceedings of the Findings of the Association for Computational Linguistics: ACL 2023, 2023, pp. 13003–13051.
49. Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; Steinhardt, J. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874* **2021**.
50. Jin, D.; Pan, E.; Oufattole, N.; Weng, W.H.; Fang, H.; Szolovits, P. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences* **2021**, *11*, 6421.
51. Jin, Q.; Dhingra, B.; Liu, Z.; Cohen, W.; Lu, X. Pubmedqa: A dataset for biomedical research question answering. In Proceedings of the Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP), 2019, pp. 2567–2577.
52. Fansi Tchango, A.; Goel, R.; Wen, Z.; Martel, J.; Ghosn, J. Ddxplus: A new dataset for automatic medical diagnosis. *Advances in neural information processing systems* **2022**, *35*, 31306–31318.
53. Johnson, A.E.; Pollard, T.J.; Greenbaum, N.R.; Lungren, M.P.; Deng, C.y.; Peng, Y.; Lu, Z.; Mark, R.G.; Berkowitz, S.J.; Horng, S. MIMIC-CXR-JPG, a large publicly available database of labeled chest radiographs. *arXiv preprint arXiv:1901.07042* **2019**.
54. Valmeekam, K.; Marquez, M.; Olmo, A.; Sreedharan, S.; Kambhampati, S. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. *Advances in Neural Information Processing Systems* **2023**, *36*, 38975–38987.
55. Liu, Y.; Iter, D.; Xu, Y.; Wang, S.; Xu, R.; Zhu, C. G-eval: NLG evaluation using gpt-4 with better human alignment. In Proceedings of the Proceedings of the 2023 conference on empirical methods in natural language processing, 2023, pp. 2511–2522.
56. Lee, Y.; Kim, J.; Kim, J.; Cho, H.; Kang, J.; Kang, P.; Kim, N. Checkeval: A reliable llm-as-a-judge framework for evaluating text generation using checklists. In Proceedings of the Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025, pp. 15782–15809.
57. Tian, S.Y.; Zhou, Z.; Yu, K.Y.; Yang, M.; Jia, L.H.; Guo, L.Z.; Li, Y.F. VCSearch: Bridging the Gap Between Well-Defined and Ill-Defined Problems in Mathematical Reasoning. In Proceedings of the Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025, pp. 12721–12742.
58. Kitchner, K.S. Cognition, metacognition, and epistemic cognition: A three-level model of cognitive processing. *Human development* **1983**, *26*, 222–232.
59. Schraw, G.; Dunkle, M.E.; Bendixen, L.D. Cognitive processes in well-defined and ill-defined problem solving. *Applied cognitive psychology* **1995**, *9*, 523–538.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.